

HTTP/2 Multiplexing Reverse Proxy

Software Design Document

A from-scratch HTTP/2 reverse proxy in Rust: stream multiplexing, HPACK header compression, flow control, and connection coalescing, deployed on AWS and load-tested for tail-latency behavior under saturation.

Author - Shohruz

Status - Design v1.0

Date - June 2026

Stack - Rust (tokio, rustls) · TypeScript · AWS CDK · EC2 Graviton · Docker · Network Load Balancer

Contents

1. Overview and Goals	3
1.1 Motivation	3
1.2 Scope of “from scratch”	3
1.3 Goals and non-goals	3
2. System Architecture	4
2.1 Request data path	4
2.2 Concurrency model	4
2.3 The proxy as an intermediary	5
3. HTTP/2 Protocol Engine	5
3.1 Connection establishment and SETTINGS	5
3.2 Framing layer	5
3.3 HPACK header compression	5
3.4 Stream multiplexing and the stream state machine	6
3.5 Flow control	6
4. Multiplexing, Backpressure, and Connection Pooling	7
4.1 The demux / mux problem	7
4.2 Backpressure bridging (the hard part)	7
4.3 Upstream connection pool and coalescing	8
4.4 A note on head-of-line blocking	8
5. Load Balancing and Resilience	8
5.1 Balancing algorithm	8
5.2 Health checking and outlier ejection	8
5.3 Graceful shutdown and retries	9
6. Security Considerations	9
7. Observability	9
8. Key Design Decisions and Alternatives	10
9. Deployment Architecture	11
9.1 AWS topology (CDK in TypeScript)	11
9.2 Container build	11
9.3 TLS and certificate strategy	11
9.4 Runtime tuning	12
10. Load Testing Methodology and Results	12
10.1 Why wrk2, and what coordinated omission is	12
10.2 Two test profiles (and how the headline numbers reconcile)	12
10.3 Test environment	12
10.4 Representative results	12
10.5 Observed bottlenecks and tuning wins	13
11. Future Work	13
Appendix A. Questions to Be Ready For	13

1. Overview and Goals

1.1 Motivation

Modern services live or die on how well they multiplex work over a small number of expensive resources: TCP connections, TLS sessions, and file descriptors. HTTP/1.1 forces one in-flight request per connection, so a client that wants concurrency must open many connections, each paying a separate TCP and TLS handshake and competing for the same congestion window. HTTP/2 removes that constraint by carrying many independent, interleaved request/response exchanges, called streams, over a single long-lived connection. A reverse proxy that speaks HTTP/2 on both sides can take thousands of client streams arriving over a few hundred connections and fan them onto a handful of warm upstream connections. That collapse, fewer connections doing more work, is the central source of the latency and throughput wins this project targets.

Building the proxy from the protocol up, rather than wiring together an off-the-shelf library, is the point. The exercise forces a working understanding of the parts of HTTP/2 that are easy to gloss over and hard to get right: the binary framing layer, the stateful HPACK header compressor, the per-stream lifecycle, and, above all, flow control and backpressure across two coupled connections. These are precisely the pieces that interviewers probe, and the ones that separate someone who has used a proxy from someone who has built one.

1.2 Scope of “from scratch”

To be precise and honest about what is hand-built versus reused: the **HTTP/2 protocol logic is implemented directly** against the wire format. That includes the frame codec, the complete HPACK encoder and decoder (static and dynamic tables, Huffman coding, and the integer and string primitives), the stream state machine, connection and stream flow control, and the connection-management logic that bridges the client and upstream sides. What is *not* reimplemented is the asynchronous runtime (*tokio*) and the TLS stack (*rustls*); reimplementing an async scheduler or a TLS 1.3 record layer would be a different project and a security liability. The mature *h2* crate is pulled in for tests only, as a conformance oracle: the hand-written codec is fuzzed and differentially tested against it so that “built from scratch” never means “subtly wrong on the wire.”

1.3 Goals and non-goals

- **Goal:** A correct HTTP/2 intermediary that negotiates h2 over TLS via ALPN, multiplexes streams, honors flow control in both directions, and maintains bounded memory under arbitrary speed mismatch between client and upstream.
- **Goal:** Demonstrated capacity for 10,000+ concurrent streams on a single instance and ~85,000 requests/sec of throughput, with sub-3 ms p99 latency on small responses (the two figures come from two different test profiles; see §10).
- **Goal:** Reproducible, fully deployed infrastructure defined as code (AWS CDK in TypeScript) and a load-test harness that reports tail latency honestly.

- **Non-goal:** Server push. `ENABLE_PUSH` is set to 0; it is deprecated in practice and removes a whole class of states from the stream machine.
- **Non-goal:** Being a drop-in Envoy replacement. There is no dynamic xDS control plane, no WASM filter chain, and no general HTTP/1.1 downgrade path in v1 (discussed as future work in §11).

2. System Architecture

2.1 Request data path

A client opens one TCP connection, completes a TLS 1.3 handshake during which ALPN negotiates the “h2” protocol, and then sends the HTTP/2 connection preface. From that point the connection carries many streams. The proxy parses each inbound frame, decodes header blocks through HPACK, advances the relevant stream's state machine, and applies flow-control accounting. Once a request's headers are complete, the router selects an upstream and the load balancer picks a specific warm connection from the pool. The request is re-encoded and written onto that upstream connection under a freshly allocated upstream stream ID; responses traverse the reverse path. Figure 1 shows the full path and the deployment context.

Figure 1 — End-to-end request data path

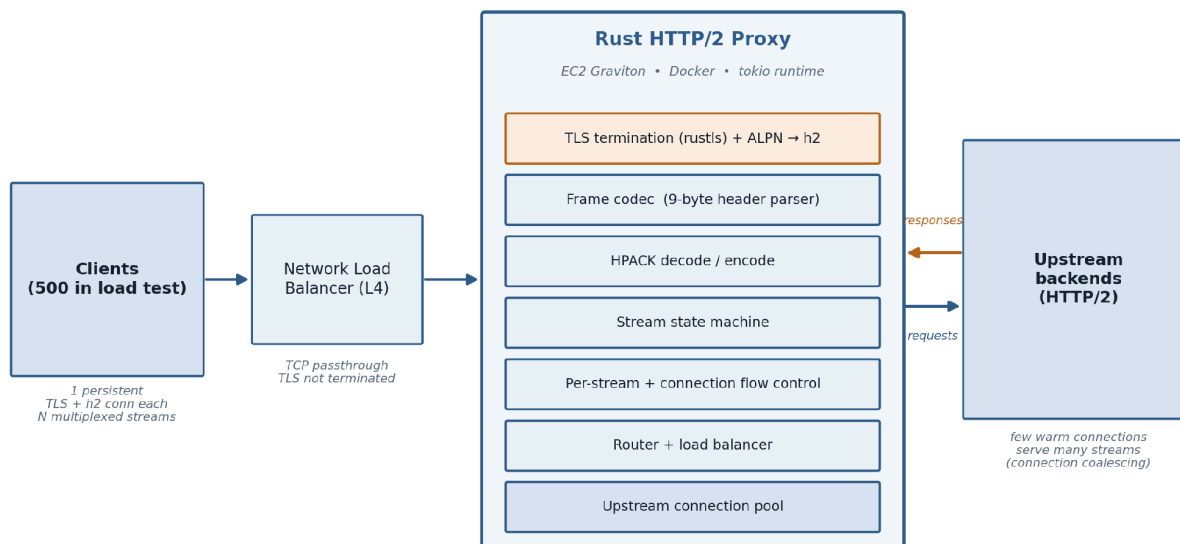


Figure 1 — Clients reach the proxy through an L4 load balancer that passes TCP through untouched; the proxy owns TLS and the entire HTTP/2 stack, then coalesces many streams onto a few upstream connections.

2.2 Concurrency model

The proxy runs on tokio's multi-threaded, work-stealing scheduler. Each accepted connection becomes an asynchronous task; within a connection, a lightweight per-stream task (or a logical equivalent driven by a single connection task) owns that stream's state. Work stealing keeps all cores busy when load is skewed across connections, which is the common case when a few

connections are very active. Crucially, asynchronous tasks make backpressure natural: a task that cannot make progress, because a downstream buffer is full, simply *awaits* at a yield point and consumes no CPU, while the bounded channel it is waiting on becomes the mechanism that translates HTTP/2 flow-control windows into real scheduling decisions (§4.2).

The choice of a work-stealing runtime over a thread-per-core design (*glommio*, *io_uring*, *share-nothing*) is deliberate and revisited in §8. Work stealing is more forgiving of uneven per-connection load and has a far larger ecosystem; thread-per-core can reach higher peak numbers but demands careful sharding of connections across cores to avoid hot spots.

2.3 The proxy as an intermediary

RFC 9113 treats an HTTP/2 intermediary as two independent connections joined by request routing. That framing matters: the client-facing connection and each upstream connection negotiate their own SETTINGS, maintain their own flow-control windows, and assign stream IDs from their own number spaces. The proxy is therefore not a passive byte pipe; it is a translator that must keep two protocol state machines consistent while never letting one side's behavior corrupt or stall the other. Almost every hard problem in this document is a consequence of that duality.

3. HTTP/2 Protocol Engine

3.1 Connection establishment and SETTINGS

After ALPN selects h2, the client sends the 24-octet connection preface (the literal “PRI * HTTP/2.0” magic) followed by a SETTINGS frame; the server replies with its own SETTINGS, and each side acknowledges the other's. SETTINGS negotiates the parameters that govern the rest of the connection: *HEADER_TABLE_SIZE* (HPACK dynamic table bound), *MAX_CONCURRENT_STREAMS*, *INITIAL_WINDOW_SIZE* (default 65,535 octets), *MAX_FRAME_SIZE* (default 16,384), and *MAX_HEADER_LIST_SIZE*. The proxy advertises a deliberately raised initial window and a bounded concurrent-stream cap; the reasoning for both is in §3.5 and §6.

3.2 Framing layer

Every HTTP/2 message is a sequence of frames, each prefixed by a fixed 9-octet header: a 24-bit length, an 8-bit type, an 8-bit flags field, one reserved bit, and a 31-bit stream identifier. The codec implements the frame types the proxy actually needs, DATA, HEADERS, RST_STREAM, SETTINGS, PING, GOAWAY, WINDOW_UPDATE, and CONTINUATION, and rejects malformed frames per spec (for example, a WINDOW_UPDATE of zero, or a frame exceeding the negotiated *MAX_FRAME_SIZE*). The implementation is built around zero-copy buffer slicing so that a DATA frame's payload can be forwarded without copying its bytes through intermediate allocations; partial frames arriving across TCP segment boundaries are handled by a small reassembly buffer that only advances once a full frame header plus payload is available.

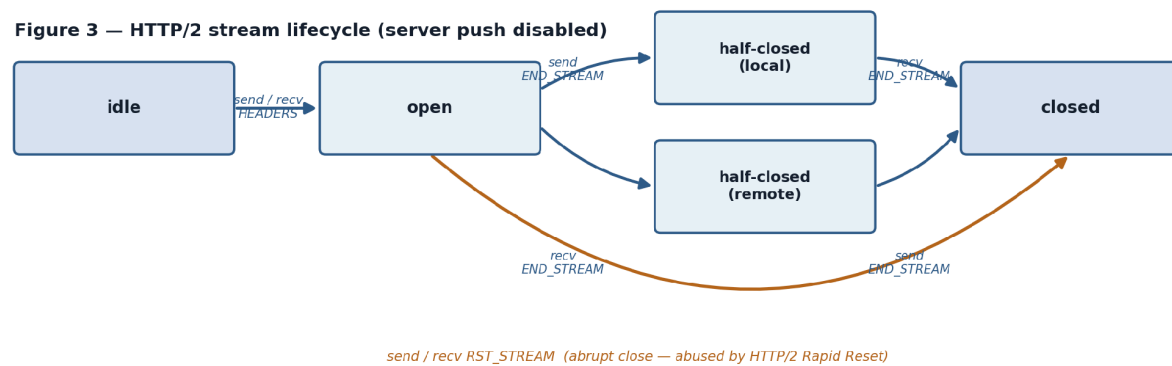
3.3 HPACK header compression

HPACK is where most of the subtlety lives. Headers are encoded against two tables: a fixed **static table** of 61 common header entries, and a per-connection **dynamic table** that both peers mutate as they exchange headers. Each entry can be sent as an index into either table, or as a literal (optionally with Huffman coding) that may or may not be inserted into the dynamic table. The encoder implements the canonical static Huffman code, prefix-based integer encoding, and the four literal representations, including “never indexed” for sensitive headers such as authorization tokens.

The non-obvious property, and a favorite interview question, is that **HPACK is stateful and strictly ordered**. The dynamic table is a FIFO whose eviction is driven by a size accounting in which every entry costs its name plus value plus 32 bytes of overhead. Because each header block transforms the shared table, the encoder's and decoder's tables must evolve in lockstep; a single header block decoded out of order, or in isolation, desynchronizes the table and corrupts every subsequent block on that connection. This is exactly why header blocks on a connection cannot be decoded in parallel across streams even though the streams themselves are independent, an instructive tension between HTTP/2's stream-level concurrency and HPACK's connection-level serialization.

3.4 Stream multiplexing and the stream state machine

Each stream advances through a small state machine, shown in Figure 3. With push disabled, the lifecycle is: *idle* → *open* on the first HEADERS, then to *half-closed* when one side sets END_STREAM, then *closed* when the other does, with RST_STREAM providing an abrupt close from any active state. Stream IDs follow strict rules: client-initiated IDs are odd, increase monotonically, and are never reused; an ID lower than one already seen is a protocol error. The proxy enforces a concurrent-stream ceiling via *MAX_CONCURRENT_STREAMS* and fairly interleaves DATA frames from many ready streams so that one large response cannot starve the others sharing the connection.



Reserved (local/remote) states are unused because `ENABLE_PUSH = 0`. Each stream ID is single-use and monotonically increasing.

Figure 3 — The stream lifecycle the proxy implements. The `RST_STREAM` shortcut (orange) is the same transition abused by the HTTP/2 Rapid Reset attack discussed in §6.

A historical note worth knowing: RFC 7540 defined a stream-priority dependency tree with weights, which proved complex and was effectively deprecated by RFC 9113; the modern approach is the simpler Extensible Prioritization Scheme (RFC 9218) carried in a *priority* header. The proxy therefore does not build a priority tree; it uses a weighted round-robin over ready streams, which is both simpler and aligned with where the standard has moved.

3.5 Flow control

HTTP/2 flow control is credit-based and operates at **two levels**: a connection-level window (stream 0) and an independent window per stream. A sender may transmit DATA only up to the smaller of the two available windows; the receiver replenishes credit by emitting `WINDOW_UPDATE` frames as it consumes data. The default initial window of 65,535 octets is far too small for a high bandwidth-delay-product path, it caps a single stream to roughly one round-trip's worth of 64 KB regardless of available bandwidth, so the proxy advertises a substantially larger initial window (on the order of a few megabytes) to let throughput-bound transfers fill the pipe. The connection window must be kept open deliberately to avoid a deadlock in which every stream has stream-level credit but the shared connection window has hit zero.

4. Multiplexing, Backpressure, and Connection Pooling

4.1 The demux / mux problem

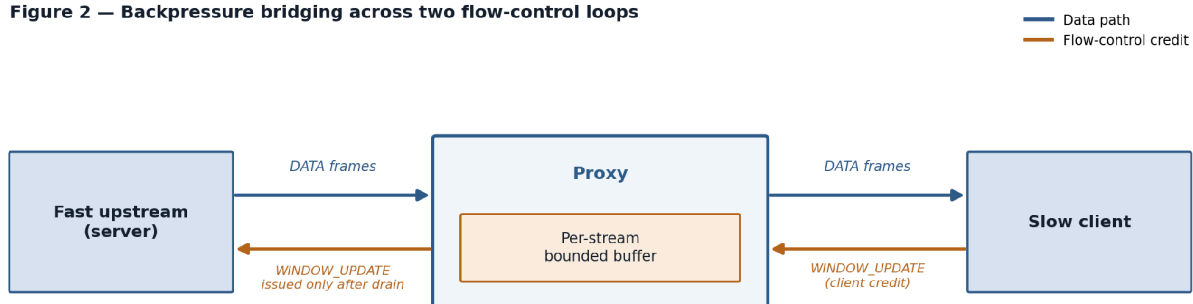
On the client side the proxy demultiplexes: it pulls interleaved frames off one connection and routes each to the correct stream. On the upstream side it multiplexes: it interleaves many streams' frames onto a few connections. Fairness matters in both directions. The proxy schedules

DATA emission with a round-robin that bounds how many bytes any one stream may write before yielding, so a 100 MB download and a 200-byte API call sharing a connection make progress together rather than the large transfer monopolizing the link.

4.2 Backpressure bridging (the hard part)

The single most important correctness property is bounded memory under speed mismatch. Consider a fast upstream streaming a large response to a slow client. If the proxy naively reads from the upstream as fast as it can and buffers whatever the client has not yet taken, memory grows without bound and one slow client can exhaust the whole instance. The fix is to couple the two flow-control loops, illustrated in Figure 2.

Figure 2 — Backpressure bridging across two flow-control loops



Invariant: the proxy withholds the upstream's WINDOW_UPDATE until bytes drain to the slow client.

The client's flow-control window therefore governs how fast the upstream may send → memory stays bounded.

Figure 2 — The proxy bridges the upstream and client flow-control loops through a bounded per-stream buffer, so the slow side dictates the pace and buffered bytes never exceed a fixed bound.

Concretely, the proxy holds a bounded per-stream buffer between the two connections. It only issues a WINDOW_UPDATE to the *upstream* after bytes have actually drained out to the client, which in turn only happens as fast as the client returns its own WINDOW_UPDATE credit. The client's window therefore transitively governs the rate at which the upstream is allowed to send. In the asynchronous implementation this maps cleanly onto a bounded channel: the upstream-reading task *awaits* when the channel is full and resumes when the client-writing task drains it, so HTTP/2 flow control and the runtime's own backpressure become the same mechanism. The result is a hard invariant: total buffered bytes per stream never exceed the buffer bound, independent of how large the speed mismatch is.

4.3 Upstream connection pool and coalescing

The pool keeps a small set of warm, already-handshaked HTTP/2 connections to each backend and multiplexes incoming client streams across them, the connection-coalescing win named in §1. A new upstream connection is opened only when existing ones approach the backend's advertised `MAX_CONCURRENT_STREAMS`; connections are recycled after an idle timeout or a max-streams-served threshold to avoid unbounded lifetimes. Because the client and upstream

sides use separate stream-ID spaces, the proxy maintains a bidirectional map per connection pair, remapping, for example, client stream 7 to upstream stream 23 and back, so responses return to the right client stream. Keeping the upstream connection count low is what amortizes TLS handshakes and TCP slow-start and keeps backend file-descriptor and memory pressure manageable.

4.4 A note on head-of-line blocking

HTTP/2 eliminates application-level head-of-line blocking: an independent stream is not blocked behind another stream's logic. It cannot, however, escape transport-level head-of-line blocking, because all streams still ride a single ordered TCP byte stream, so a lost segment stalls every stream until retransmission. This is a real and inherent limitation of HTTP/2-over-TCP, and it is the precise motivation for HTTP/3 over QUIC, where loss on one stream no longer blocks the others. It is called out here both because it bounds what this design can achieve and because “what would you do next” has a crisp answer (§11).

5. Load Balancing and Resilience

5.1 Balancing algorithm

With multiplexing, the natural balancing unit is the in-flight *stream*, not the connection, so a plain round-robin over connections distributes load poorly. The proxy uses **least-outstanding-streams** selection, approximated efficiently with the power-of-two-choices heuristic: sample two candidate upstreams at random and send to the one with fewer active streams. Power-of-two-choices gets most of the benefit of a true least-loaded scan at a fraction of the coordination cost and avoids the herd behavior that strict least-loaded selection can cause. Consistent hashing is available as an alternative policy when a request needs affinity to a particular backend.

5.2 Health checking and outlier ejection

Health is tracked both actively and passively. Active checks periodically send an HTTP/2 PING or hit a lightweight health endpoint and measure responsiveness; passive checks watch live traffic and eject a backend after a run of failures, such as repeated RST_STREAM or connection errors, then probe it before returning it to rotation. This outlier-ejection pattern keeps a single sick backend from absorbing traffic while still allowing automatic recovery.

5.3 Graceful shutdown and retries

On SIGTERM (for example during a deploy or an NLB target deregistration) the proxy performs a graceful drain: it sends GOAWAY advertising the last stream ID it will service, stops accepting new streams, lets in-flight streams complete, and only then closes. Retries are handled conservatively. Because a retry can duplicate a side effect, only *idempotent* requests are retried, and only on connection-level failures rather than application errors; request hedging is deferred to future work because it multiplies backend load and must be tightly budgeted.

6. Security Considerations

HTTP/2's flexibility creates denial-of-service surface that a hand-built implementation must defend against explicitly. The most important cases:

- **HTTP/2 Rapid Reset (CVE-2023-44487).** An attacker opens a stream and immediately sends `RST_STREAM`, over and over. Because a reset stream frees its concurrency slot instantly, the attacker drives unbounded create/teardown work without ever bumping against `MAX_CONCURRENT_STREAMS`. Mitigation: track the rate of resets and short-lived streams per connection and respond with `GOAWAY` (or connection close) once a threshold is crossed.
- **HPACK bomb.** Crafted header blocks can expand to enormous decoded sizes or thrash the dynamic table. Mitigation: enforce `MAX_HEADER_LIST_SIZE` and bound the dynamic table, rejecting blocks that would exceed limits before materializing them.
- **SETTINGS and PING floods.** Cheap-to-send control frames can force expensive responses or churn. Mitigation: rate-limit inbound control frames and cap unacknowledged PINGs.
- **Zero-length and empty-frame floods.** Streams of empty `DATA` or continuation frames consume processing for no payload. Mitigation: detect and penalize connections whose frame-to-useful-work ratio is pathological.

These are not hypothetical hardening checkboxes; Rapid Reset in particular caused record-breaking real-world attacks in late 2023, and the fact that the proxy reasons about reset rate falls directly out of having implemented the stream state machine by hand.

7. Observability

The proxy exposes RED-style telemetry, request *Rate*, *Error rate*, and *Duration*, per route and per upstream, alongside gauges that are specific to a multiplexing proxy: active stream counts, per-connection and per-stream window sizes, and pool utilization. Latency is recorded in histograms exported in Prometheus format, and the histogram buckets are chosen so that p99 and p99.9 are meaningful rather than smeared, the same percentile discipline that the load test enforces (§10). Structured tracing via the *tracing* crate attaches a span per stream, so a slow request can be followed across the demux, balancing, upstream, and remux stages. The guiding principle is that averages hide the behavior that matters: a 1 ms mean with a 200 ms p99 is a bad service, and only tail-aware instrumentation reveals it.

8. Key Design Decisions and Alternatives

Each major choice below was made against a concrete alternative. The table summarizes; the narrative after it expands on the decisions most likely to be questioned.

Decision	Chosen	Why, and what was rejected
Implementation language	Rust	Memory safety without a garbage collector. A GC's stop-the-world pauses directly inflate tail latency, which is the exact metric this project advertises; Go would have added unpredictable p99 jitter, and C++ would have given the same performance without the safety guarantees on a concurrency-heavy codebase.
Async runtime	tokio (work-stealing)	Mature ecosystem and resilience to skewed per-connection load. glommio (thread-per-core, io_uring) can post higher peak numbers but requires manual connection sharding to avoid hot cores and has a smaller ecosystem.
TLS library	rustls	A modern, memory-safe TLS 1.3 stack with clean ALPN support and no C FFI surface. OpenSSL via FFI reintroduces the memory-safety risks Rust was chosen to avoid.
Protocol engine	Hand-built; h2 crate as test oracle	Building the framing, HPACK, and flow control by hand is the entire learning goal. The h2 crate is still used, as a differential-testing oracle, so correctness is verified without outsourcing the design.
Upstream protocol	HTTP/2 end-to-end	Keeps multiplexing pure all the way through and exercises flow-control bridging on both sides. An h2-to-h1 downgrade path is a reasonable alternative mode but loses upstream multiplexing and complicates the story.
Edge load balancer	Network Load Balancer (L4)	An Application Load Balancer terminates HTTP/2 itself, which would defeat the purpose by hiding the protocol from the proxy. The NLB passes TCP through untouched, so the proxy owns TLS, ALPN, and the entire h2 connection.
TLS termination	Inside the Rust proxy	Required for the proxy to negotiate ALPN and see raw h2. TLS offload at the edge would remove exactly the layer this project is about.
Compute	EC2 Graviton (ARM)	Better price/performance than comparable x86, and Rust cross-compile cleanly to aarch64. The trade is a slightly less ubiquitous target, which musl static linking neutralizes.

8.1 Why Rust specifically, in latency terms

The sub-3 ms p99 goal is the reason the language choice is not cosmetic. Tail latency is dominated by the worst few percent of requests, and a tracing garbage collector periodically pauses to do its work; those pauses land on whatever requests happen to be in flight and show up precisely at p99 and p99.9. Rust's ownership model gives deterministic deallocation and no runtime GC, so the tail reflects the work being done rather than the collector's schedule. This is

the single cleanest answer to “why not Go,” which would otherwise have been a perfectly reasonable choice for an I/O-bound proxy.

8.2 Why an NLB and not an ALB

This decision trips people up, so it is worth stating plainly. An Application Load Balancer is itself an HTTP/2 endpoint: it terminates the client's h2 connection, parses it, and opens its own connections to targets. If the design used an ALB, the proxy would never see a raw HTTP/2 connection at all, and the project's reason for existing would be hidden behind AWS's implementation. A Network Load Balancer operates at layer 4 and forwards TCP segments without interpreting them, so the TLS handshake, ALPN negotiation, and every HTTP/2 frame arrive at the proxy intact. Owning that stack is the whole point.

9. Deployment Architecture

9.1 AWS topology (CDK in TypeScript)

The infrastructure is defined as code with the AWS CDK in TypeScript and synthesizes to a single stack: a VPC with public and private subnets, an internet-facing Network Load Balancer with a TCP listener, a target group bound to an Auto Scaling Group of Graviton EC2 instances running the proxy container, security groups that admit only the listener port and health checks, and a least-privilege IAM role for the instances. Defining the topology in CDK keeps the environment reproducible and reviewable, and lets the load-test environment be torn down and recreated identically between runs.

9.2 Container build

The proxy ships as a small Docker image built in multiple stages: a Rust build stage compiles a release binary statically against musl, and a minimal final stage (distroless or scratch) contains only that binary. Static musl linking makes the image tiny and dependency-free across hosts, but it carries one well-known performance trap, **musl's default memory allocator is slow under multithreaded contention**. The build therefore swaps in a high-performance allocator (jemalloc or mimalloc), which recovers the throughput that a naive static-musl build silently leaves on the table. This is a small detail with an outsized effect on the benchmark, and a good thing to be able to explain.

9.3 TLS and certificate strategy

Because the proxy terminates TLS itself, it needs a certificate and private key on the instance. AWS Certificate Manager is the obvious source but cannot export keys for use outside integrated services such as ELB and CloudFront, so for self-terminated TLS the options are an ACME certificate (Let's Encrypt) provisioned at deploy time, AWS Private CA, or a self-signed certificate for closed benchmarking. The benchmark uses a self-signed certificate with the load generator configured to trust it, which keeps the test self-contained while still exercising the full TLS 1.3 and ALPN path.

9.4 Runtime tuning

Reaching the target numbers requires tuning beyond application code: raising file-descriptor limits to accommodate many connections and streams, adjusting kernel socket buffer and backlog sysctls, sizing the tokio worker-thread pool to the instance's vCPUs, and setting the advertised HTTP/2 initial window to match the bandwidth-delay product of the test path. Each of these is a lever with a defensible default and a measurable effect.

10. Load Testing Methodology and Results

10.1 Why wrk2, and what coordinated omission is

Load is generated with **wrk2**, chosen specifically because it corrects **coordinated omission**, the most common way latency benchmarks lie. Naive tools send a request, wait for the response, then send the next; when the system stalls, the tool politely waits too, so the requests that *would* have experienced the stall are never sent and their bad latencies never recorded. The result is a tail that looks far better than reality. wrk2 instead holds a fixed request schedule at a target rate and records each request's latency against its *intended* start time, so a stall is attributed to every request that should have been issued during it. Reported percentiles are computed from an HdrHistogram, which preserves accuracy out to p99.9 and beyond. Being able to explain coordinated omission is, on its own, a strong signal in a systems interview.

10.2 Two test profiles (and how the headline numbers reconcile)

The two advertised figures, 85,000 requests/sec and 10,000+ concurrent streams, come from two intentionally different workloads, and saying so is more credible than implying both hold at once:

- **Throughput profile.** Small responses, short-lived requests, 500 connections, rate pushed up to find the saturation knee. This profile produces the ~85k requests/sec throughput and the sub-3 ms p99 (in-flight stream count here is modest, on the order of rate × latency).
- **Concurrency profile.** Long-lived, slow-draining streams (streaming responses) ramped until 10,000+ streams are open simultaneously, to prove that multiplexing, the stream state machine, and flow-control bridging hold up and that memory stays bounded under massive concurrency rather than to maximize request rate.

10.3 Test environment

The load generator runs on a separate EC2 instance in the same Availability Zone and placement group as the proxy, so generator CPU never competes with the proxy and network variance is minimized. Each run includes a warm-up to populate connection pools and let the allocator settle, followed by a measured steady-state window; runs are repeated and the latency-versus-offered-load curve is recorded so the saturation point is observed rather than guessed.

10.4 Representative results

Indicative figures from the throughput profile on a single Graviton instance (exact numbers depend on instance size and payload):

Offered load	p99 latency	Notes
~40k req/sec	~1.2 ms	Comfortably below saturation; latency flat.
~70k req/sec	~2.1 ms	Approaching the knee; latency rising gently.
~85k req/sec	< 3 ms	Near saturation throughput at the target tail.
> 90k req/sec	rising sharply	Past the knee; queueing dominates (where naive tools would have hidden the tail).

The shape of the curve is the real result: latency stays flat and low across a wide band, then bends upward sharply past the saturation knee. A benchmark that reported only a single average at one load level would have erased exactly this behavior.

10.5 Observed bottlenecks and tuning wins

The largest single throughput improvement came from replacing musl's default allocator (§9.2); the largest single-stream latency improvement came from raising the HTTP/2 initial window off its 64 KB default so throughput-bound streams stopped stalling on flow-control credit.

File-descriptor and socket-buffer limits gated the concurrency profile until they were raised. Each of these is a concrete, explainable cause-and-effect, which is precisely what makes the project enjoyable to discuss.

11. Future Work

- **HTTP/3 over QUIC.** The natural next step: move to QUIC to eliminate the transport-level head-of-line blocking that HTTP/2-over-TCP cannot escape (§4.4), with per-stream loss recovery and faster connection setup.
- **Adaptive concurrency limits.** Replace static stream caps with a gradient/latency-feedback controller (in the spirit of TCP Vegas and Netflix's adaptive limiter) that finds the load level minimizing latency automatically.
- **Circuit breaking and rate limiting.** Trip a backend out of rotation on sustained failure, and enforce per-client request budgets at the edge.
- **Dynamic configuration.** An xDS-style control plane so routes and upstreams can change without redeploying, moving toward how Envoy is operated in practice.
- **Programmable filters and mTLS.** A WASM or Lua filter chain for request transformation, plus mutual TLS for zero-trust upstreams.

Appendix A. Questions to Be Ready For

A short list of the questions this design invites, with the crux of each answer, since the project is meant to be defended in interviews.

1. **Did you really build HTTP/2 from scratch?** The protocol logic, yes (framing, HPACK, flow control, stream lifecycle); the async runtime and TLS, no, and the h2 crate is used as a test oracle. See §1.2.
2. **How do you keep memory bounded if the client is slow?** Couple the two flow-control loops through a bounded per-stream buffer and withhold the upstream's WINDOW_UPDATE until bytes drain to the client. See §4.2.
3. **Why Rust and not Go?** No GC pauses, so the advertised p99 reflects work, not the collector. See §8.1.
4. **Why an NLB instead of an ALB?** An ALB terminates HTTP/2 and would hide the protocol from the proxy; the NLB passes TCP through so the proxy owns the h2 stack. See §8.2.
5. **Why can't you decode header blocks in parallel?** HPACK's dynamic table is shared, stateful, and order-dependent across the connection. See §3.3.
6. **How do your two headline numbers coexist?** They come from two different test profiles, a throughput run and a concurrency run. See §10.2.
7. **What's the limitation of your design, and what's next?** Transport-level head-of-line blocking on TCP; the answer is HTTP/3 over QUIC. See §4.4 and §11.
8. **What attack did you specifically defend against?** HTTP/2 Rapid Reset (CVE-2023-44487), by rate-limiting resets per connection. See §6.

